

① $E \rightarrow E + E \mid id$

soln

$$E \rightarrow E + E$$

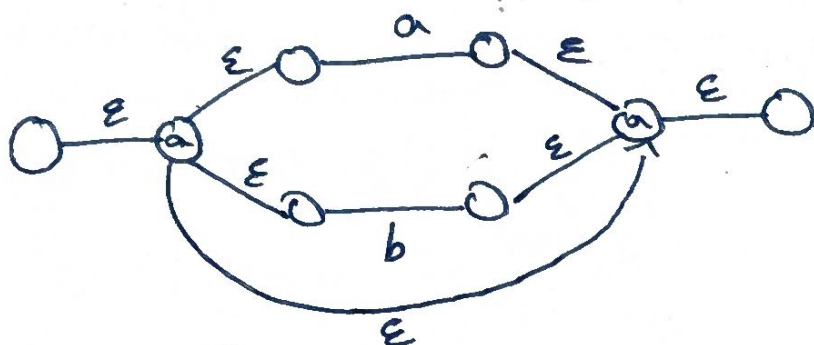
$$E \rightarrow id$$

It is ambiguous, because left side and right side both have the same ~~term~~-terminal.

40

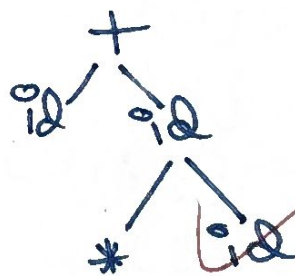
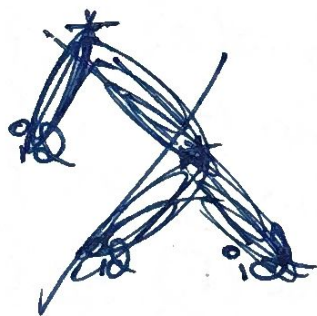
100

② (a) $a(a|b)^*a$



③ $id + id * id$

Parse tree:



④ left recursion From the grammar $A \rightarrow A\alpha / B$

$$A \rightarrow A\alpha / B$$

$$A \rightarrow A\alpha$$

$$A \rightarrow B$$

$$A \rightarrow BA'$$

$$A' \rightarrow \alpha A$$

⑤

$$S \rightarrow iETS | iETSES | a$$

Soln

$$S \rightarrow iETS | iETSES | a$$

By applying iET factoring

$$ex: A \rightarrow \alpha B / \alpha$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \beta / \alpha$$

then,

$$S \rightarrow \beta S'$$

$$S' \rightarrow \beta / \alpha$$

2

Given gram

Soln

left sec

⑥

$$S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bc / E$$

$$D \rightarrow EF$$

$$E \rightarrow g / E$$

$$F \rightarrow F / E$$

$$\text{First}(S) : \{a\}$$

$$\text{First}(B) : \{c\}$$

$$\text{First}(C) : \{b\}$$

$$\text{First}(D) : \{g, F\}$$

$$\text{First}(E) : \{g\}$$

$$\text{First}(F) : \{F\}$$

$$\text{Follow}(S) : \{\$ \}$$

$$\text{Follow}(B) : \{g, F, \$ \}$$

$$\text{Follow}(C) : \{g, F, \$ \}$$

$$\text{Follow}(D) : \{g, F, \$ \}$$

$$\text{Follow}(E) : \{F, g, \$ \}$$

$$\text{Follow}(F) : \{F, g, \$ \}$$

3

I.

To F

⑨

Given grammar: $S \rightarrow Sa | Sb | cd$.

Soln

left recursion:

$$S \rightarrow Sa | Sb | cd$$

$$S \rightarrow Sa$$

$$S \rightarrow Sb$$

$$S \rightarrow cd$$

$$S \rightarrow bs'$$

$$s' \rightarrow as'$$

$$S \rightarrow as' | bs' | \epsilon$$

It contains immediate left recursion.

⑧ To find First of (E')

$$E \rightarrow TE', E' \rightarrow +TE' | \epsilon$$

Soln

$$\text{First}(E') \Rightarrow \{+, \epsilon\}$$

⑨ To check whether the string is valid.

$$S \rightarrow (s)S | \epsilon$$

(10) what actions are performed on shift reduce parser

- * Shift
- * Reduce
- * ~~Accept~~

Shift: when the element in the stack is less than the expression, then we shift to transfer the element from LHS to RHS.

Reduce: when the LHS & RHS elements have its LHS and the element is greater than the element.

Accept: when we check the previous element and got the correct output.

(11) To construct the LR(0) and identify the closure of statement.

Grammar: $S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow cC \mid d$

Item: $[S' \rightarrow \cdot S]$

Closure

$S' \rightarrow \cdot S$

$S \rightarrow \cdot CC$

$S \rightarrow \cdot CC \mid \cdot d$

$\therefore$ The closure is constructed //

(12) Given

$S \rightarrow AB, A \rightarrow a, B \rightarrow b.$

grammar $S \rightarrow aSb \mid ab$ and to check whether $aabb$ is accepted.

02/10/20
Soln

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

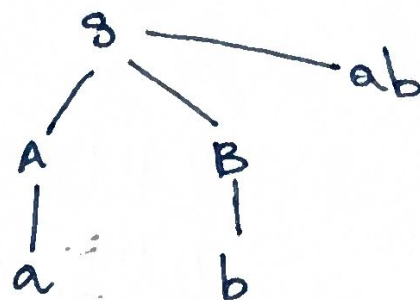
$$\text{First}(S) : \{a\}$$

$$\text{First}(A) : \{a\}$$

$$\text{First}(B) : \{b\}$$

Therefore $aabb$ is not accepted.

Parse tree



⑬ To calculate the first and follow for the grammar.

Given:

$$S \rightarrow A$$

$$A \rightarrow aB/Ad$$

$$B \rightarrow b$$

$$c \rightarrow g$$

$$A \rightarrow aBA'$$

$$A' \rightarrow dA'$$

Soln

$$\text{First}(S) : \{a\}$$

$$\text{First}(A) : \{a\}$$

$$\text{First}(A') : \{d\}$$

$$\text{First}(B) : \{b\}$$

$$\text{First}(c) : \{g\}$$

$$\text{Follow}(S) : \{\$ \}$$

$$\text{Follow}(A) : \{b, \$ \}$$

$$\text{Follow}(A') : \{b, \$ \}$$

$$\text{Follow}(B) : \{b, \$ \}$$

$$\text{Follow}(c) : \{g, b, \$ \}$$

⑭ simple lex code to parse expressions with addition code.

LDF : R_1, R_2
ADF : A_1, R_1, R_2
STF : Q, R_1

⑮
 $S \rightarrow AB$
 $A \rightarrow aA \mid \epsilon$
 $B \rightarrow b \mid c$

$\text{First}(S) : \{a\}$
 $\text{First}(A) : \{a, \epsilon\}$
 $\text{First}(B) : \{b, c\}$

$\text{Follow}(S) : \{\$ \}$
 $\text{Follow}(A) : \{\$ \}$
 $\text{Follow}(B) : \{\$ \}$

⑰ Left Recursion

Left recursion is a situation in the grammar where the non-terminal calls itself first in its own production.

Rule :

If we have a terminal we can write as it is but if we have a non-terminal then we have to eliminate the left recursing elements.

Grammar

$E \rightarrow TE'$
 $E' \rightarrow +TE' / \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' / \epsilon$
 $F \rightarrow (E) / id$

First(E) : { (, id }
First(E') : { +, ε }
First(T) : { (, id }
First(T') : { *, ε }
First(F) : { (, id }

Follow(E) : { \$,) }
Follow(E') : { \$,) }
Follow(T) : { +, \$,) }
Follow(T') : { +, \$,) }
Follow(F) : { *, +, \$,) }

Terminals

Non Terminals	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE' / \epsilon$			✓ ?	✓ ?
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'			$T' \rightarrow *FT' / \epsilon$	?	?	!
F	$F \rightarrow (E) / id$			$F \rightarrow (E) / id$		

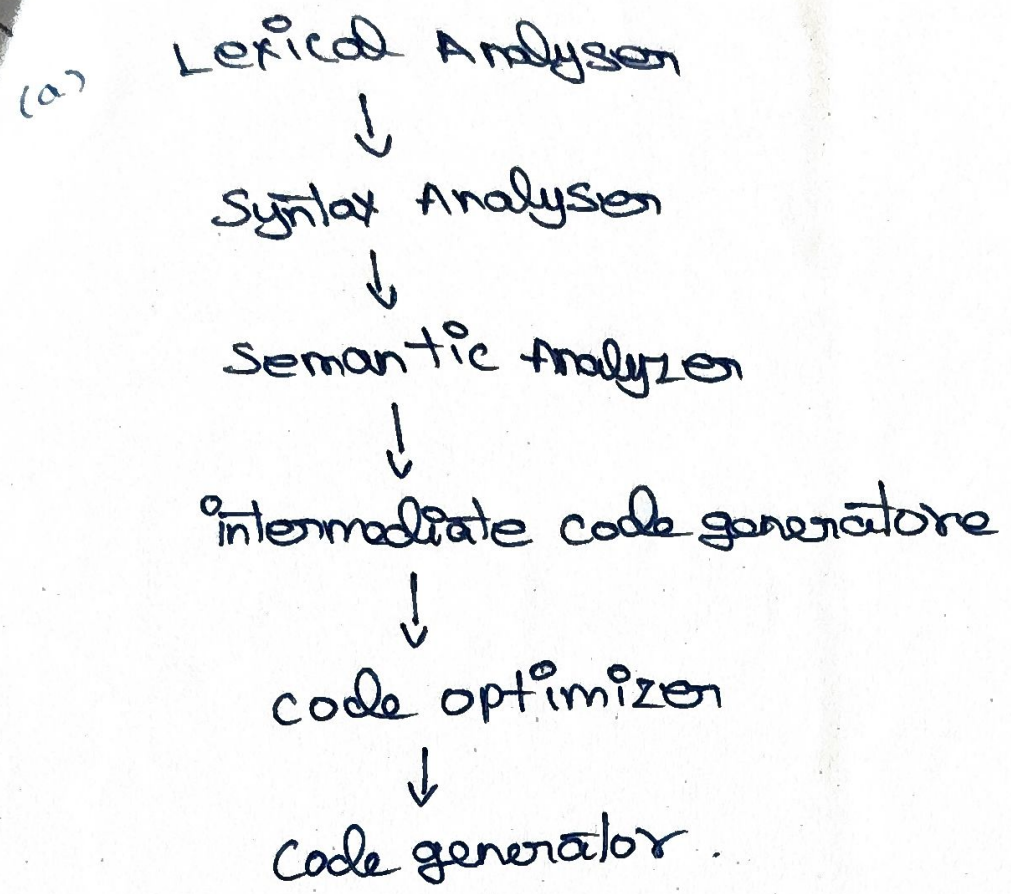
shift reduce parsing

stack	Input	output
\$	id + id * id \$	$E \rightarrow TE'$
\$ E' T	id + id * id \$	$T \rightarrow FT'$
\$ E' T' F	id + id * id \$	$F \rightarrow id$
\$ E' id	id + id * id \$	$T' \rightarrow \epsilon$
\$ E'	id + id * id \$	$E' \rightarrow +TE'$
\$ E' T		
\$ E' E' T	id * id \$	$T \rightarrow FT'$
\$ E' E' T'	id * id \$	$F \rightarrow id$
\$ E' E' T' F	* id \$	$T' \rightarrow *FT'$
\$ E' E' T' id		$E' \rightarrow \epsilon$
\$ E' E' T' F *	id \$	$F \rightarrow id$
\$ T' F	id \$	$T' \rightarrow \epsilon$
\$ T' id	\$	
\$	\$	

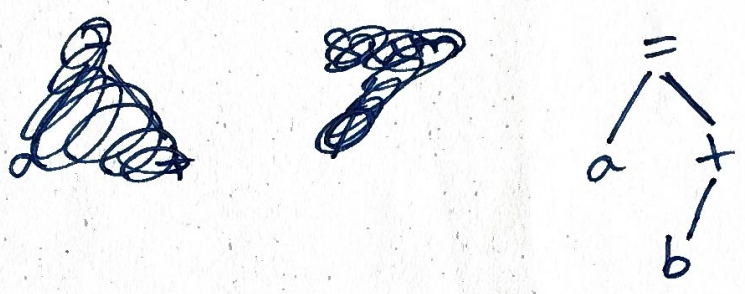
(20)

code:

```
#include <stdio.h>
int main() {
    int a=10, b=20;
    int sum = a+b;
    printf("%d", sum);
    return 0;
}
```

(b) Parse tree



(c) Given,
 $Sum = a + b$
 $S \rightarrow id = E$
 $E \rightarrow E + T \mid T \Rightarrow E \rightarrow TE'$
 $E' \rightarrow ++TE'$
 $T \rightarrow id \mid num //$

4